

需直接输入程序名即可。

例如，在 Windows 下执行 `abc.exe`，可以进入它所在目录后直接输入 `abc` 并按 Enter 键。系统为什么能找到 `abc.exe` 呢，因为在 Windows 下，当前目录是最先搜寻可执行文件的位置，并且扩展名 `.exe` 在搜索之前会被自动添加。如果当前目录没有 `abc.exe`，是否会报错呢？不一定。运行 `path` 命令，会看到一连串目录。如果当前目录没有 `abc.exe`，系统会继续在这些目录中寻找，全部查找完毕仍没找到时才会报错。在搜索文件时并不会检查上述目录下的子目录。

Linux 有一些不同。首先，它的可执行文件名并不是以 `.exe` 为扩展名的，因此 `g++ abc.cpp -o abc` 编译出的文件是 `abc`，而非 `abc.exe`（当然，如果一定要将其取名为 `abc.exe`，也无不可）。另外，当前目录并不在搜索路径中，因此，即使 `abc` 已经在当前目录中，仍需要用 `./abc` 这样的方式告诉 Linux “可执行文件 `abc` 就在当前目录”。

A.1.4 重定向和管道

很多比赛要求选手直接读写标准输入输出（即用 `printf/scanf` 或 `cin/cout` 读写，且不用 `freopen`），难道在评分时裁判要将输入数据一一用键盘输入，等程序运行结束之后看着屏幕，逐个对照手中的标准答案吗？当然不是。可以使用重定向的技巧将输入文件塞到程序的标准输入中，然后再将程序输出保存在文件中。

在 Windows 下可以使用 `abc < abc.in > abc.out`。而在 Linux 下则可以使用 `./abc < abc.in > abc.out`。当然，如果可执行文件和输入输出文件不在同一个目录，则需要进行相应调整。但基本方法是不变的：在输入文件名前面加一个 `<` 符号，而在输出文件名前面加一个 `>` 符号。注意，此时的输出文件将被覆盖。如果希望只是把输出附加在文件末尾，则可用 `>>` 代替 `>`。此外，如果有大量的文本输出到标准错误输出，还可以用 `2>` 将它们重定向，但需注意，尽量不要在正式提交的程序中输出到标准错误输出，这样不仅可能会违反比赛规定，还可能会因为大量文本的输出而占用宝贵的 CPU 资源，甚至导致超时。

Windows 和 Linux 均提供“管道”机制，用于把不同的程序串起来。例如，如果有一个程序 `aplusb` 从标准输入读取两个整数 a 和 b ，计算并输出 $a+b$ ，还有一个程序 `sqr` 从标准输入读取一个整数 a ，计算并输出 a^2 ，则可以这样计算 $(10+20)^2$ ：`echo 10 20 | aplusb | sqr`。尽管也可以用重定向来完成这个任务，但用管道明显要简单得多。

另一个常见用法是分页显示一个文本文件的内容。在 Windows 下可以用 `type abc.txt | more`，在 Linux 下则是用 `cat abc.txt | more`。

A.1.5 常见命令

在 Linux 中，可以用 `time` 命令计时。例如，运行 `time ./abc` 会执行 `abc` 并输出运行时间。但 Windows 中并没有这样的命令，幸好在大多数情况下只是在对自己编写的程序计时，因此只需在程序的最后打印出 `clock() / (double)CLOCKS_PER_SEC` 即可（需要包含 `time.h`）。

附表 A-1 中给出了一些常见命令的 Linux 版本和 Windows 版本，供读者查阅。